SAHAYAK Master Guide

For Johan Jyoti Kuli - Lead Developer and Project Head

This guide explains the whole project in plain language. Use it to understand the system, introduce the project, explain your own work honestly, and answer judge questions. You do not need to memorise every line of code, but you should know what each part does and how the parts fit together.

1. Opening speech

Good morning respected judges, teachers, and everyone here. My name is Johan Jyoti Kuli, and I am the lead developer and project head of SAHAYAK. SAHAYAK is a real-time Indian Sign Language recognition prototype. It uses a camera, hand-point detection, a trained model and a website to predict a limited set of sign labels. I will explain how a hand becomes numbers, how the model makes a guess, how the website and server share information, and how we test and improve the system.

2. What problem are we exploring?

People who use sign language may meet someone who does not understand their signs. SAHAYAK explores whether a computer can recognize some trained signs and show their labels. Indian Sign Language is a full language with its own rules. Recognizing a small set of hand signs is not the same as understanding a full conversation. SAHAYAK is not a universal translator and is not a replacement for a trained interpreter.

3. Simple technical words

Computer vision: using software to find information in images or video.
Landmark: a key point on the hand, such as a fingertip or joint.
Feature: a number that describes something useful about the hand.
Dataset: examples with the correct label written beside each one.
Training: showing labelled examples to a program so it can learn patterns.
Model: the saved result of training.
Prediction: the model’s best guess for a new example.
Frontend: the website pages and controls that a user sees.
Backend: the server that receives information and runs the prediction.
API: a web address used by the website to send information to the server.
Candidate model: a trial model that is not active yet.
Promote: approve the trial model and make it active.
Deploy: publish approved files to the hosted site or server.
Confidence: a score showing how strongly the model favours a label. It is not a promise that the answer is right.

4. Follow one prediction from start to end

1. Camera: the browser asks the user for camera permission and shows the video.
2. Hand points: MediaPipe Hands finds key points on visible hands in the browser.
3. Feature numbers: JavaScript prepares numbers that describe hand shape and position.
4. API request: the browser sends those numbers to the backend.
5. Model: the Python Flask backend loads the saved model and asks it to predict a label.
6. Response: the backend sends the label and any available confidence information back.
7. Display: the website shows the result. Translation or speech may happen separately.

Remember the order: CAMERA -> HAND POINTS -> FEATURE NUMBERS -> BACKEND API -> MODEL -> PREDICTED LABEL -> WEBSITE.
If the points look correct but the label is wrong, that does not mean the same step failed. Hand tracking and sign prediction are separate parts.

5. Why 126 static values?

A static sign is a pose that can be described in one frame. MediaPipe estimates 21 key points per hand. Each point has x, y and z coordinates. One hand has 21 x 3 = 63 values. Two hand slots

have 63 x 2 = 126 values. The model uses these values to choose a label. The number 126 is not the number of signs, the number of samples or the accuracy.

6. Why 1,320 values for a moving sign?

A moving sign depends on how the hands change over time. Each sequence frame has 132 values: 126 hand-point values plus six wrist-position values. The system arranges the sequence into 10 time steps. 132 x 10 = 1,320 values for the sequence model. This describes the model input format; it does not prove the model knows every moving sign.

7. What each part does

- HTML gives the website its page structure.
- CSS controls its look and layout.
- JavaScript handles buttons, camera preview, hand-point drawing, feature numbers and API requests.
- MediaPipe Hands finds hand points in browser frames.
- Python and Flask run the backend API.
- The scikit-learn-compatible model saved by Joblib predicts sign labels.
- SQLite can save continuous-sign samples in the training project.
- Vercel hosts the website frontend.
- Render runs the backend and loads the saved models.
- A service worker can save website files in the browser cache. This does not make live backend predictions work fully offline.

Do not describe the current backend as a TensorFlow or CNN system unless the code and model files are changed and checked. Explain what is actually in the project.

8. Important API routes

/api/health - checks whether the backend responds. It does not measure accuracy.
/api/status - shows model status and current model labels.
/api/predict-features - receives static numeric features from the current Live Studio workflow.
/api/predict - an older or alternate image route. Check the frontend before saying it is used in the current workflow.
/api/sequence/predict - receives continuous-sequence features.
/api/sequence/samples and /api/sequence/export - support continuous sample access as coded.
/api/translate and /api/speech - support translation or speech. Some services need internet access.

When explaining a route, know what the browser sends and what the backend returns. Check frontend/studio.html and backend/web_app.py.

9. Add and train a static sign

1. Back up the current CSV, active model and label file.
2. Collect correctly labelled poses in data/landmarks_2hand.csv.
3. Check that old labels and the new label are still present. Rows use a label and values v0 through v125.
4. Train a trial model using train_model.py.
5. Check that the candidate has 126 features and all expected labels.
6. Run python -m unittest test_sahayak.py.
7. Test with fresh examples that were not used for training.
8. If the results are acceptable, run python train_model.py --promote.
9. Test old and new signs in the local website.
10. Back up the deployed copy and copy only the active static model.
11. Commit only the intended model file, push it, wait for Render, check /api/status and test the live site.

Do not train on only the new sign if the model must keep recognizing old signs. The candidate needs examples for every class that should remain.

10. Add and train a continuous sign

1. Back up continuous_gesture_db.db and the active sequence model.

2. Use collect_sequences.py to save the full movement under one stable label.
3. Check label spelling and sample counts.
4. Train a trial model using python train_sequence_model.py --db.
5. Check the 1,320-value input and expected labels.
6. Record a fresh test set after training.
7. Review the overall score and the score for each label.
8. If acceptable, run python train_sequence_model.py --db --promote.
9. Run python -m unittest test_sahayak.py test_sequence.py test_sequence_db.py.
10. Test old and new signs locally.
11. Copy only sahayak_sequence.joblib to the deployment project.
12. Commit only the intended model file, wait for Render, check sequence_model_ready and sequence_sign_labels, then test the live site.
Do not replace the static model during a continuous-only update. Do not publish raw training databases or samples by mistake.

11. How to test fairly

Keep test examples separate from training examples. Better still, use different recording sessions or different signers for testing. Record how many examples were tested and how many were correct for each sign. Check which signs are confused. Test different lighting, camera distance, hand positions and movement speed. Quote an accuracy percentage only when a saved test report supports it. A successful demo or fast response is not proof of accuracy.

12. Privacy and responsible use

The user should choose when to allow the camera. Explain when information is sent to the backend. Get permission from anyone who provides training examples. Keep training files safe. Do not store identifiable video or images without a clear need. Do not call this a guaranteed tool or recommend it for emergencies.

13. Deploying updates

- Website change: test the frontend, push the intended files and confirm the Vercel deployment.
- Backend code or model change: test what you can locally, push the intended files and confirm the Render deployment.
- Model-only change: normally does not require changing frontend code.
- After a backend update, check /api/health and /api/status.
- Open the live website and test old as well as new signs.
If an update fails, check Git status, the browser console, the API response and the newest hosting logs. Do not reset your work or force-push just to hide an error.

14. Quick troubleshooting

Camera does not open: check browser permission, camera selection and whether another app is using it.
Hand points do not line up: check mirroring and drawing coordinates.
Points appear but the label is wrong: check feature order, hand slots, model version, lighting and label similarity.
New label is missing: check the active model, the deployed file, the Git commit and /api/status.
Backend is unavailable: check internet, Render status and deployment logs.
Website looks old: refresh after deployment; cached files may need a hard refresh.
Old signs stop working: restore the previous model and test the candidate again.

15. Questions judges may ask

Q: What did you build? A: I led the design and integration of the website, camera and hand tracking, model API, model files and deployment workflow. I should explain my real contribution honestly.
Q: Is it a complete ISL translator? A: No. It predicts a limited set of trained labels.
Q: Why MediaPipe? A: It finds hand points that can be changed into numbers.
Q: Why 126 values? A: Two hands x 21 key points x three coordinates.

Q: Why 1,320 values? A: 132 values per frame across 10 time steps.
Q: Does adding a label file teach the model? A: No. New labelled examples, training, testing and deployment are needed.
Q: How do you prove accuracy? A: Use a saved test report with examples kept separate from training.
Q: Why separate frontend and backend? A: The browser handles the camera and display; the server loads the model and predicts.
Q: Does it work offline? A: Not fully. Live prediction and some extra services need a connection.
Q: Can it be used in emergencies? A: No. It is a limited prototype and can be wrong.
Q: What is useful about the project? A: It joins hand tracking, a trained classifier and a website into one system that can be tested.

16. Closing speech

SAHAYAK shows one way a camera, hand-point detection, a trained model and a website can work together. The current model knows only a limited set of signs and can make mistakes. The next steps are to collect more varied samples with permission, test fairly, improve weak labels and share clear results. Thank you.

Extra workflow details to know

Change website content without retraining

For a text, layout or colour change, edit the frontend files, run the page locally, check that the links and controls still work, push only the intended frontend changes and verify the Vercel deployment. You normally do not retrain the model for a words-only change. A change to feature extraction or recognition code is different: it must still match the model input format and needs proper tests.

Can GitHub train a new model?

Not automatically. GitHub saves the source code. Training on a GitHub runner requires an actual automated workflow, the training scripts, required packages, access to the correct labelled data and enough computing power. Do not claim that training runs automatically unless you have checked the workflow logs. When unsure, use the local training scripts that are present and tested.

Confidence cutoff and sequence size

The current static code accepts the top prediction when its probability is at least 0.60, where that score is available. Below the cutoff, the app may show an uncertain result. The 0.60 value is a program setting, not a statement that the model is 60 percent accurate. The continuous-sign segmenter currently uses segments of about 15 to 90 frames. Read the current code before giving these exact settings to a judge because they may change.

Saved samples and response timing

A local training database is different from the active model file. A server's local database may not be permanent unless its host has permanent storage configured. Do not promise that samples recorded on a hosted server will always remain available. Keep backups of important training data.

Recent API response times show server handling time for requests. They do not measure whether a sign prediction was correct, and the timing history can reset when the server restarts. A full camera-to-screen timing test measures more than the backend alone.

Keep proof of testing

For each version, record the model file or version, the labels tested, number of fresh examples per label, correct and incorrect results, lighting and camera conditions, and any changes made afterward. Test old labels after adding a new one. Never quote an accuracy number unless a saved test result supports it.

Final lead-developer checklist

[ ] I can explain the camera-to-result workflow. [ ] I know the static and continuous models. [ ] I can explain the feature counts. [ ] I can find the main frontend and backend files. [ ] I can check API status. [ ] I back up files before changing models. [ ] I can explain limits

honestly.